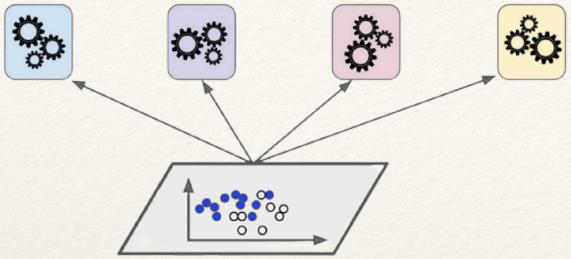
Applied Machine Learning - Advanced

Prof. Daniele Bonacorsi, Dott. Luca Clissa

Lecture 6

[tmp - 15.05.2026]

Data Science and Computation PhD + Master in Bioinformatics
University of Bologna



Ensemble Learning

"Wisdom of the crowd" (more later..)

Aggregate and average over answers from a pool of experts usually is wiser than taking the answer by just one expert

This idea in ML gives this tactic:

- run a group of predictors (such as classifiers or regressors)
- aggregate their predictions

A group of predictors is called an "**ensemble**" → these techniques are referred to as "**Ensemble Learning**" (Ensemble algos/methods)

We will discuss the most popular ensemble methods, including:

- **voting**
- **bagging**
- **boosting**
- **stacking**

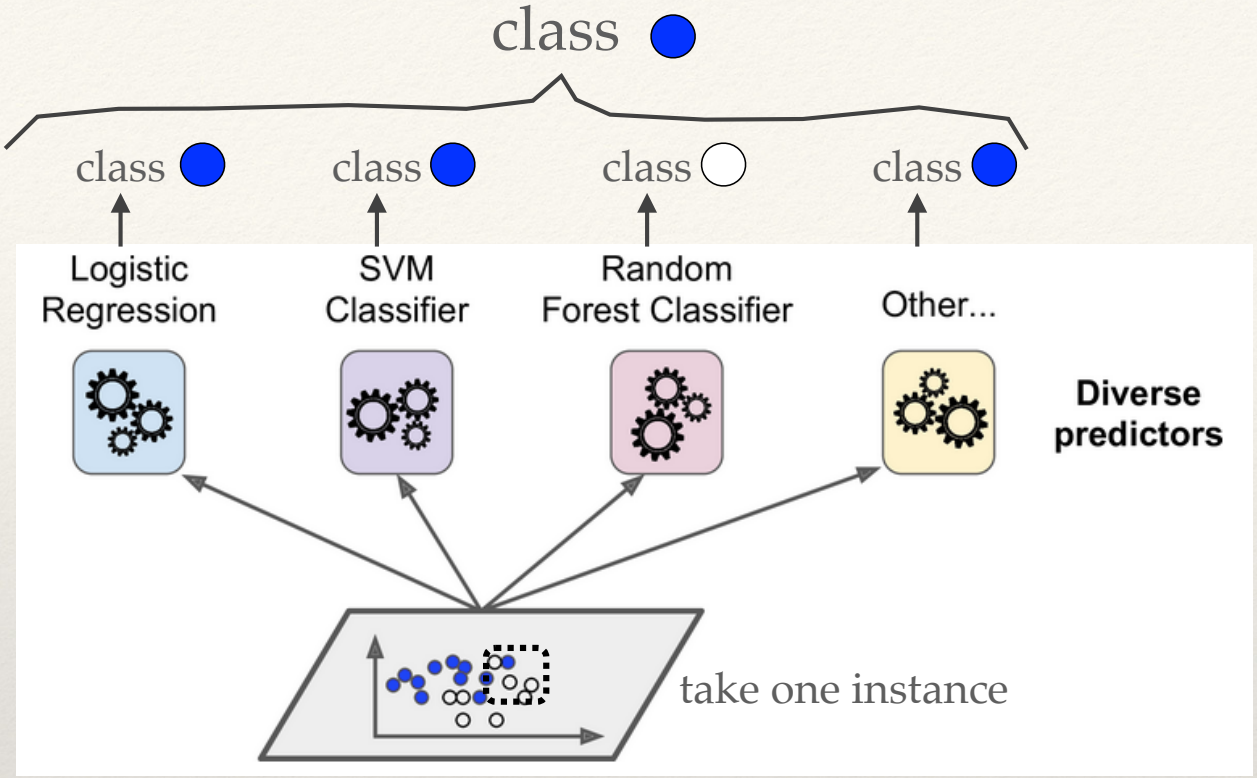
Ensemble :: Voting

Ensemble :: Bagging

Ensemble :: Boosting

Ensemble :: Stacking

(Hard) **Voting** classifiers



I trained various classifiers. Each one achieving its own (goodish) accuracy.

An even better classifier can be built by aggregating the predictions from each classifier and then predict, *per each instance*, the class that gets the most votes out of the ensemble of classifiers

- This "majority-vote" classifier is called a "**hard voting classifier**"

Wisdom of the crowd: wiser than the wisest?!

A voting classifier often achieves higher accuracy than the best classifier in the ensemble (!)

- even if each classifier is a weak learner (i.e. only slightly better than random guessing), the ensemble of classifiers can still be a strong learner (i.e. achieving high accuracy), provided 1) there are a sufficient number of weak learners and 2) they are sufficiently diverse.

Wait, what?! How is that possible?!

- Basically, law of large numbers.
- Suppose I build an ensemble containing 10^3 classifiers that are relatively poor, i.e. individually correct only 51% of the time (barely better than random guessing). If I predict the majority voted class, I can hope for something up to 75% accuracy!
- However, this is true if all classifiers are perfectly independent, make uncorrelated errors, etc (impossible, they are trained on the same data..). Best is to put together **diverse classifiers trained using very different algos**, i.e. higher chances they will make **very different types of errors**, thus **improving the ensemble's accuracy**.

An implementation in sklearn

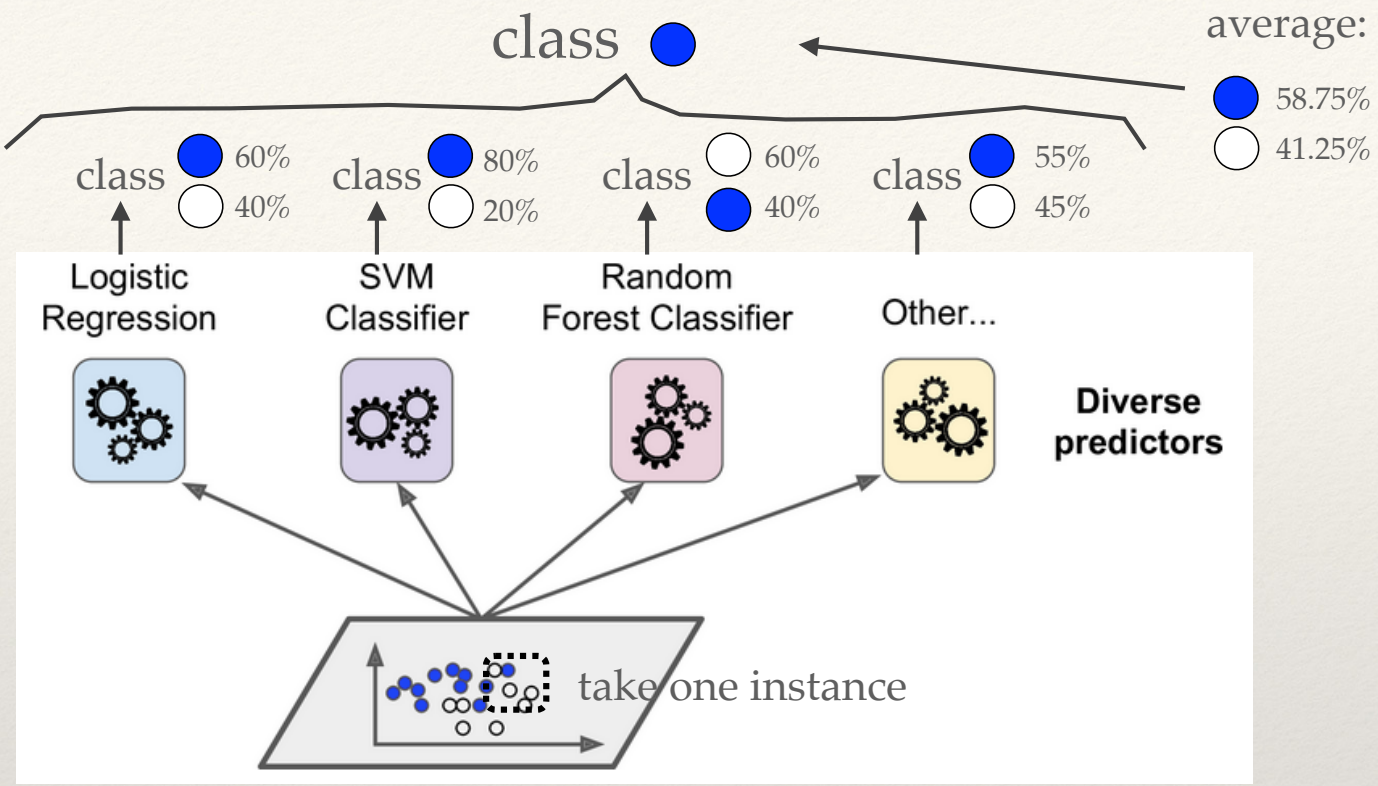
Check the code carefully..

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

log_clf = LogisticRegression()
rnd_clf = RandomForestClassifier()
svm_clf = SVC()

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='hard')
voting_clf.fit(X_train, y_train)
```

```
>>> from sklearn.metrics import accuracy_score
>>> for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
...     clf.fit(X_train, y_train)
...     y_pred = clf.predict(X_test)
...     print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
...
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.888
VotingClassifier 0.904
```

The "hard" voting classifier slightly outperforms all the individual classifiers.

(Soft) **Voting** classifiers



If (and only if) all classifiers you select for your ensemble are able to estimate class probabilities (i.e. in sklearn they have a predict_proba() method), then you can avoid a hard voting, and instead use sklearn to **predict the class with the highest class probability, averaged over all the individual classifiers**.

This is called "**soft voting**".

- It gives more weight to highly confident votes

A (modified) implementation in sklearn

Ensure that all classifiers can estimate class probabilities

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

log_clf = LogisticRegression()
rnd_clf = RandomForestClassifier()
svm_clf = SVC()

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='hard')
voting_clf.fit(X_train, y_train)
```

'soft'

SVC class can't by default, one needs to set its probability hyperparameter to 'True' (which will add a predict_proba() method at the cost of making SVC use cross-validation to estimate class probabilities, slowing down training..)

```
>>> from sklearn.metrics import accuracy_score
>>> for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
...     clf.fit(X_train, y_train)
...     y_pred = clf.predict(X_test)
...     print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
...
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.888
VotingClassifier 0.904
```

this becomes roughly 91%

The "soft" voting classifier would achieve over 91% accuracy

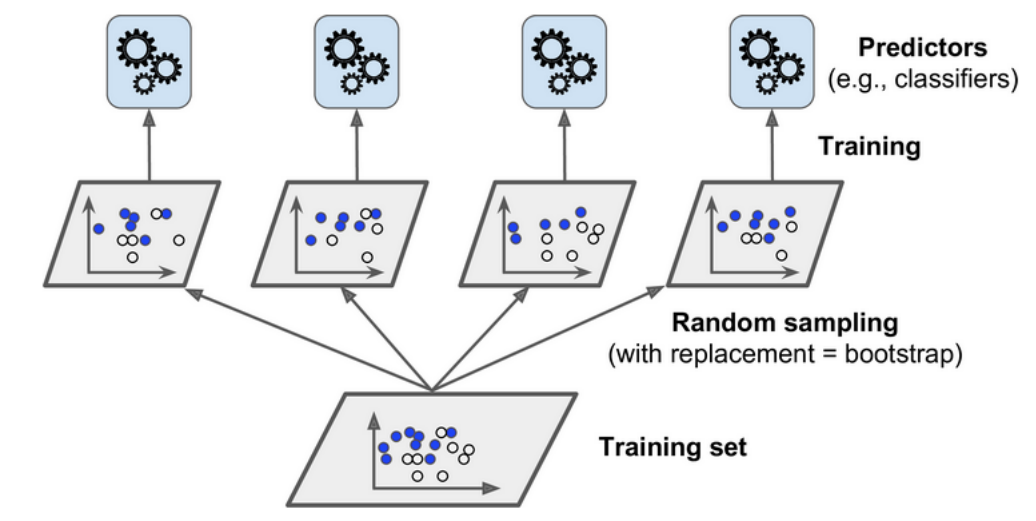
Ensemble :: Voting

Ensemble :: Bagging

Ensemble :: Boosting

Ensemble :: Stacking

Bagging (and **Pasting**)



Instead of using different training algos to get different predictors in the ensemble, just use the same training algo for every predictor, but **train them on different random subsets of the training set**.

- random sampling performed w replacement (use instances more than once) → **Bagging** (**B**ootstrap **AGG**regation → *"bootstrap=True" in the sklearn implementation*)
- random sampling performed w/o replacement (use instances only once) → **Pasting**

Aggregation of predictions from all predictors is typically the most frequent (hard voting) for classification, or the average for regression

Note: they can be trained in parallel (multiple CPUs or different servers, even), one of the reasons why they are popular: ***they scale up well****.*

Example in sklearn

BaggingClassifier automatically performs soft voting if the base classifier can estimate class probabilities

`BaggingRegressor` for regression

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    max_samples=100, bootstrap=True, n_jobs=-1)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

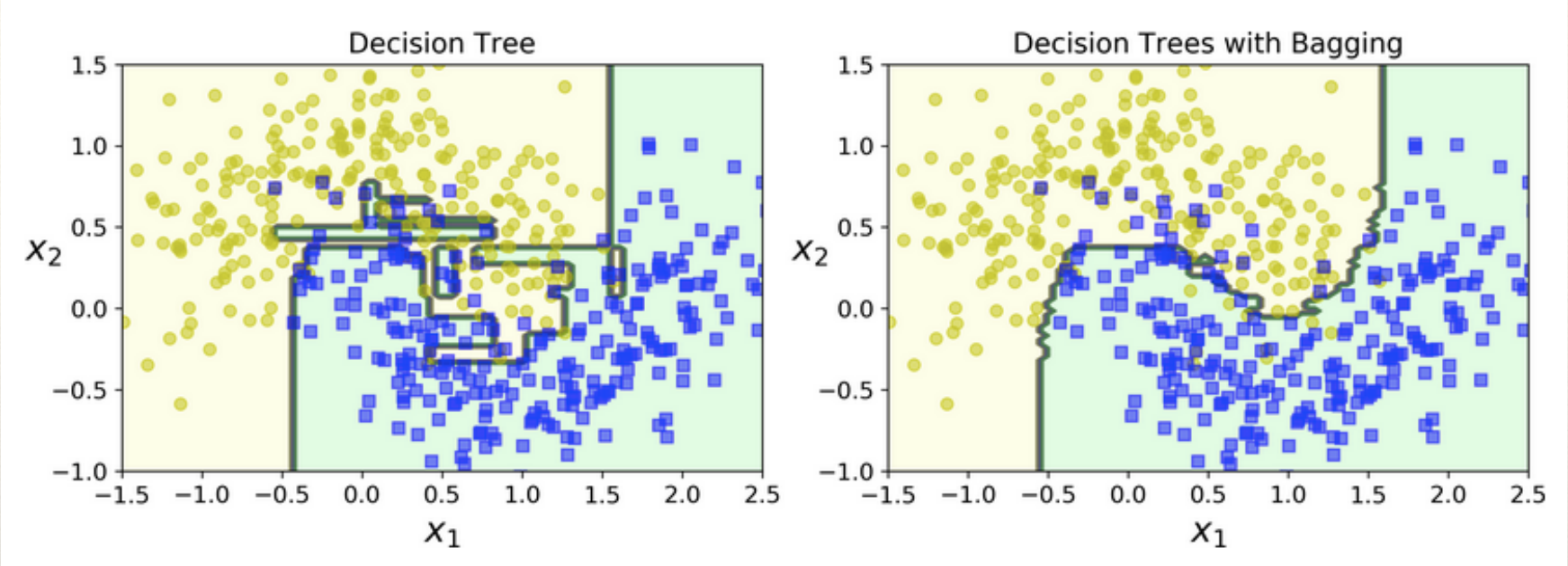
This is **Bagging**. For **Pasting**, put 'False'

This trains an ensemble of 500 DT classifiers, each trained on 100 training instances randomly sampled from the original training set with replacement

- n_jobs tells sklearn the nb CPU cores to use for training and predictions (–1 = use all available cores)

1 DT vs a bagging ensemble of 500 DT

[A]



[B]

[B]'s predictions will likely **generalize much better** than [A]

- i.e. comparable bias (roughly same nb errors on the training set)..
- .. but smaller variance (decision boundary less irregular)

This is quite general: the net result is that the ensemble has a **similar bias** but a **lower variance** than a single predictor trained on the original training set

Out-of-bag (**oob**) evaluation in bagging

With bagging, for any given predictor, some instances may be sampled several times, while others may not be sampled at all

So, **only ~63% of the m training instances are sampled on average for each predictor**

- On m training instances, bagging (i.e. w/ replacement) gives a chance for an instance of not being selected in any of draws as $(1-1/n)^n$, which for large n yields $1/e \approx 0.37$
- these **remaining 37%** are called "**out-of-bag**" (**oob**) training
  - Note that they are not the same 37% for all predictors

As a predictor never sees the oob instances during training, **it can be evaluated on these instances, without the need for a separate validation set**.

- e.g. by adding oob_score=True in my BaggingClassifier in sklearn, I can perform an oob evaluation of the ensemble's accuracy on that test set (averages out the oob evaluations of each predictor)

Random Forests

A **Random Forest** is an ensemble of DTs, generally trained via the bagging (sometimes pasting) method

- in sklearn, does this mean to build a BaggingClassifier and pass it a DecisionTreeClassifier? Yes but..
- .. one can instead use the RandomForestClassifier class (similarly: RandomForestRegressor), which is more convenient and optimised for DTs

→ Random Forest algo introduces **extra randomness when growing trees**

- instead of searching for the very best feature when splitting a node, it searches for the best feature among a random subset of features
- → greater tree diversity, which trades a higher bias for a lower variance, generally yielding an overall better model

For even additional randomness → Extremely Randomised Trees ensembles (**Extra Trees**)

- add randomness by also using random thresholds for each feature rather than searching for the best possible thresholds (like regular DTs do)
- even faster than Random Forests

Random Forests and feature importance

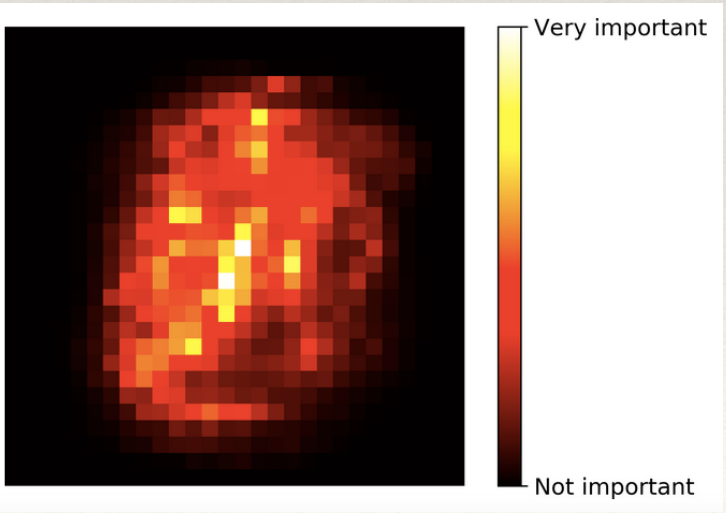
Another quality of Random Forests is that they make it easy to measure **the relative importance of each feature**

- e.g. sklearn measures a feature's importance by looking at how much the tree nodes that use that feature reduce impurity on average (across all trees in the forest)
- very useful in feature selection!

Feature importance in the IRIS dataset

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris()
>>> rnd_clf = RandomForestClassifier(n_estimators=500, n_jobs=-1)
>>> rnd_clf.fit(iris["data"], iris["target"])
>>> for name, score in zip(iris["feature_names"], rnd_clf.feature_importances_):
...     print(name, score)
...
sepal length (cm) 0.112492250999
sepal width (cm) 0.0231192882825
petal length (cm) 0.441030464364
petal width (cm) 0.423357996355
```

Pixel importance in the MNIST dataset



Ensemble :: Voting

Ensemble :: Bagging

Ensemble :: Boosting

Ensemble :: Stacking

Boosting

Boosting (originally called "hypothesis boosting") refers to any ensemble method that can combine several weak learners into a strong learner, with the general idea to **train predictors sequentially, each trying to correct its predecessor**.

There are many boosting methods available, with the by-far most popular being:

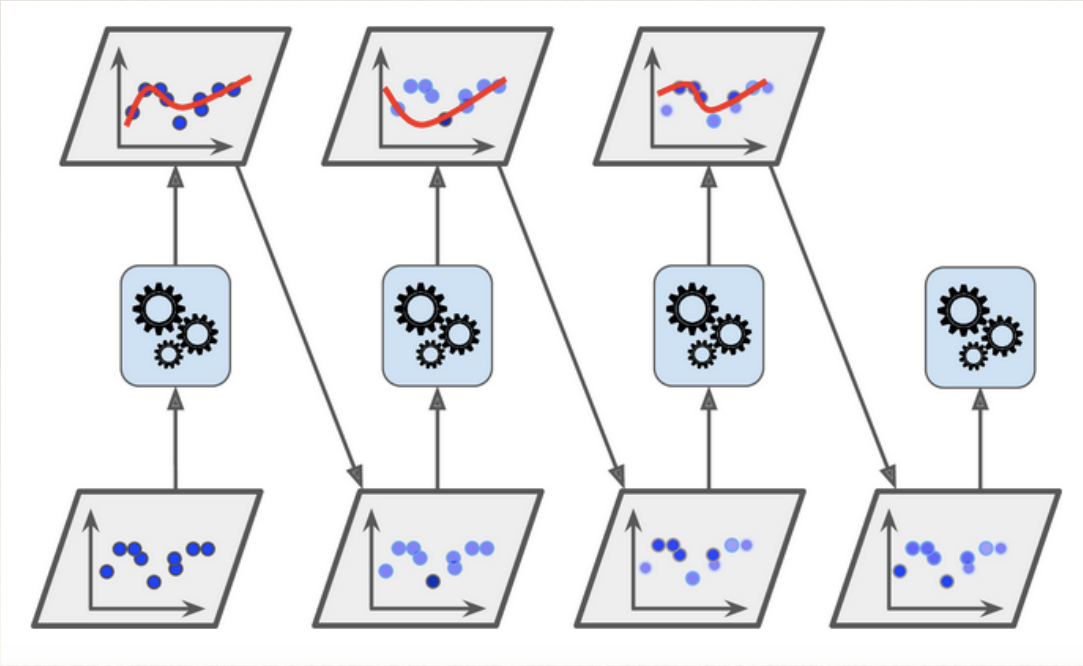
- **AdaBoost** (short for Adaptive Boosting)
- **Gradient Boosting**

AdaBoost

keyword: "adaptive"

AdaBoost is a predictor that corrects/improves its predecessor by paying a bit more attention to the training instances that the predecessor underfitted. This results in new predictors focusing more and more on the hard cases.

- E.g. a first base classifier (such as a DT) is trained and used to make predictions on the training set. The relative weight of misclassified training instances is then increased. A second classifier is trained using the updated weights and again it makes predictions on the training set, then weights are updated, and so on



Gradient Boosting

keyword: “gradient”

Popular.

Like AdaBoost, **Gradient Boosting** works by sequentially adding predictors to an ensemble, each one correcting its predecessor..

.. but ..

while AdaBoost tweaks the instance weights at every iteration - Gradient Boosting tries to **fit the new predictor to the residual errors made by the previous predictor**

- also called Gradient Tree Boosting, or Gradient Boosted Regression Trees (GBRT)

Boosting: pros and cons

Push to **high perf and high generalisation levels**, but, in general, one drawback: it is a sequential learning technique, so **it cannot be parallelised** (or only partially), since each predictor can only be trained after the previous predictor has been trained and evaluated. As a result, **it does not scale as well as bagging or pasting**.

Gradient boosting logic in sklearn (*by code*)

```
from sklearn.tree import DecisionTreeRegressor

tree_reg1 = DecisionTreeRegressor(max_depth=2)
tree_reg1.fit(X, y)
```

First, fit a DecisionTreeRegressor to the training set (e.g., a noisy quadratic)..

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2)
tree_reg2.fit(X, y2)
```

.. then train a second regressor on the residual errors made by the first predictor..

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2)
tree_reg3.fit(X, y3)
```

.. then train a third regressor on the residual errors made by the second predictor..

```
y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
```

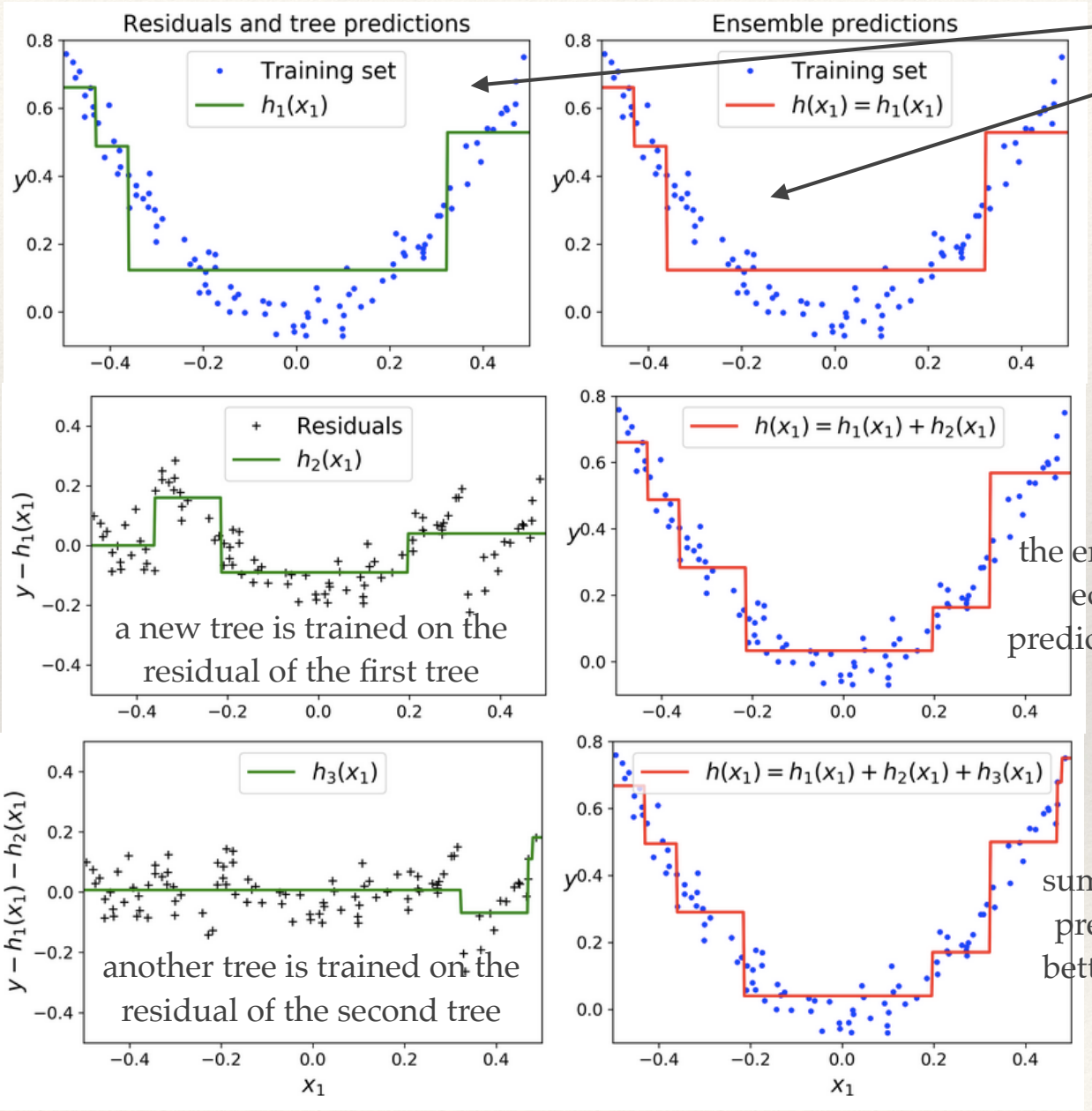
Now we have an ensemble containing 3 trees. It can make predictions on a new instance simply by adding up the predictions of all the trees

Gradient Boosting (*by visualisation*)

Prediction of tree 1

Prediction of tree 2

Prediction of tree 3



ensemble has still just 1 tree, so predictions are the same

Ensemble predictions over the sequence

a new tree is trained on the residual of the first tree

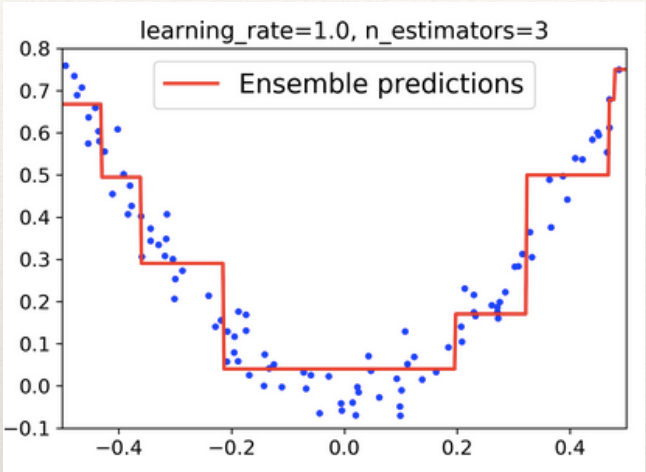
the ensemble's predictions are equal to the sum of the predictions of the first two trees

another tree is trained on the residual of the second tree

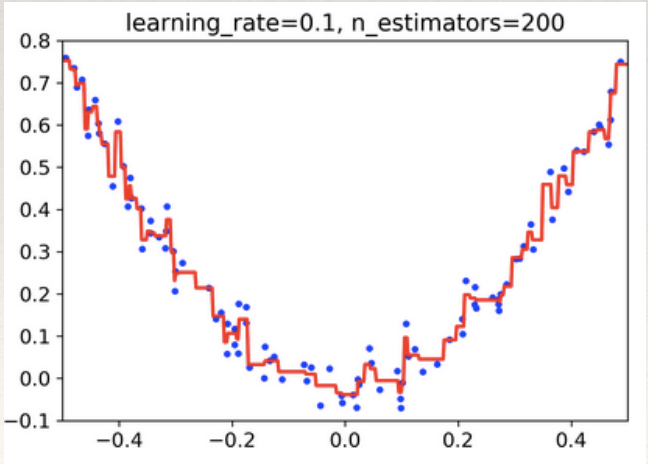
sum again. The ensemble's predictions gradually get better as trees are added to the ensemble.

Gradient Boosting tuning

Play with the learning_rate hyperparameter: it scales the contribution of each tree. If you set it to a low value (such as 0.1 in this example), you will need more trees (n_estimators) in the ensemble to fit the training set, but the predictions will usually generalise better.

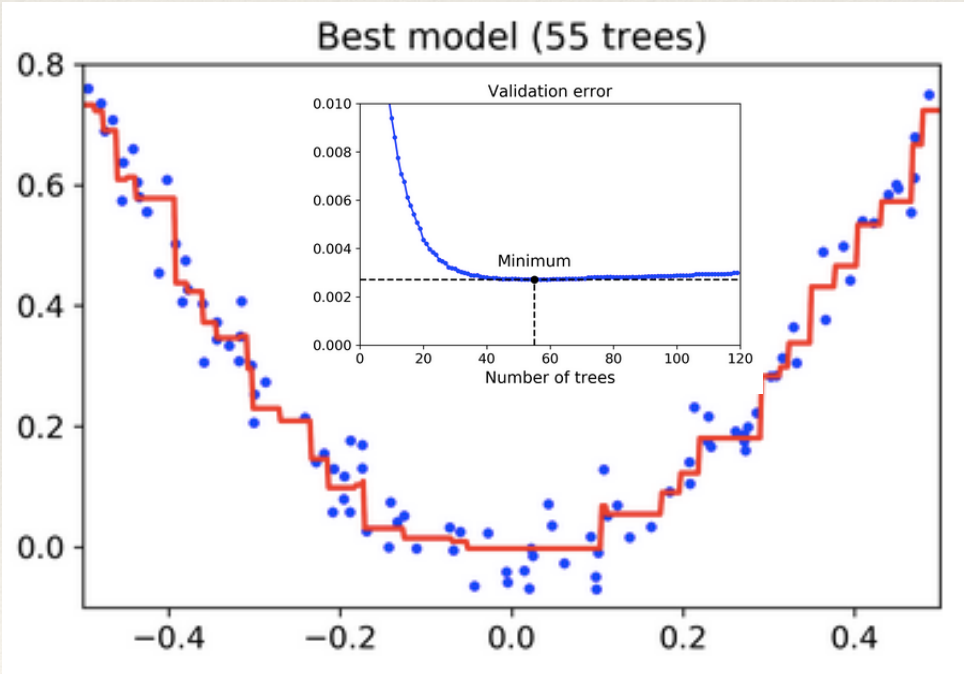


Best so far: not enough trees to fit the training set



Reduced learning rate: too many trees and overfits the training set

I can use early-stopping in sklearn: I train, while measuring the validation error at each stage of training in order to find the optimal nb trees, and once found I train another Gradient Boosting ensemble using the optimal nb trees



Extreme Gradient Boosting (**XGBoost**)

Very popular.

An optimized implementation of Gradient Boosting is available in the popular python library **XGBoost**, i.e. **Extreme Gradient Boosting**

- designed to be fast, scalable and portable

XGBoost's API is quite similar to sklearn's:

```
import xgboost

xgb_reg = xgboost.XGBRegressor()
xgb_reg.fit(X_train, y_train)
y_pred = xgb_reg.predict(X_val)
```

Some details in next slide

- keep an eye on it, as it often a key player in winning entries in ML competitions.. but it has **PLENTY** of configuration parameters, and its full coverage would take **A LOT** of time! But have a look at it if you want something powerful in your tools box..

Extreme Gradient Boosting (**XGBoost**)

XGBoost is one of the fastest implementations of gradient boosted trees. How does it work?

It directly attacks one of their major inefficiencies. It considers the potential loss for all possible splits to create a new branch (especially useful in case of thousands of features, therefore thousands of possible splits), and tackles this inefficiency by looking at the distribution of features across all data points in a leaf and using this information to reduce the search space of possible feature splits.

Yes, it uses a few regularisation tricks, but the achievable speed up is by far the most useful aspect of the XGBoost library: it allows many hyperparameter settings to be investigated quickly

- very helpful, as they are so many.. and nearly all of them are designed to limit overfitting (no matter how simple your base models are, if you put thousands of them together they will overfit..)

But…

.. its list of hyparameters is intimidating!

The list of hyperparameters is really intimidating..

- → https://xgboost.readthedocs.io/en/stable/index.html

Most important ones:

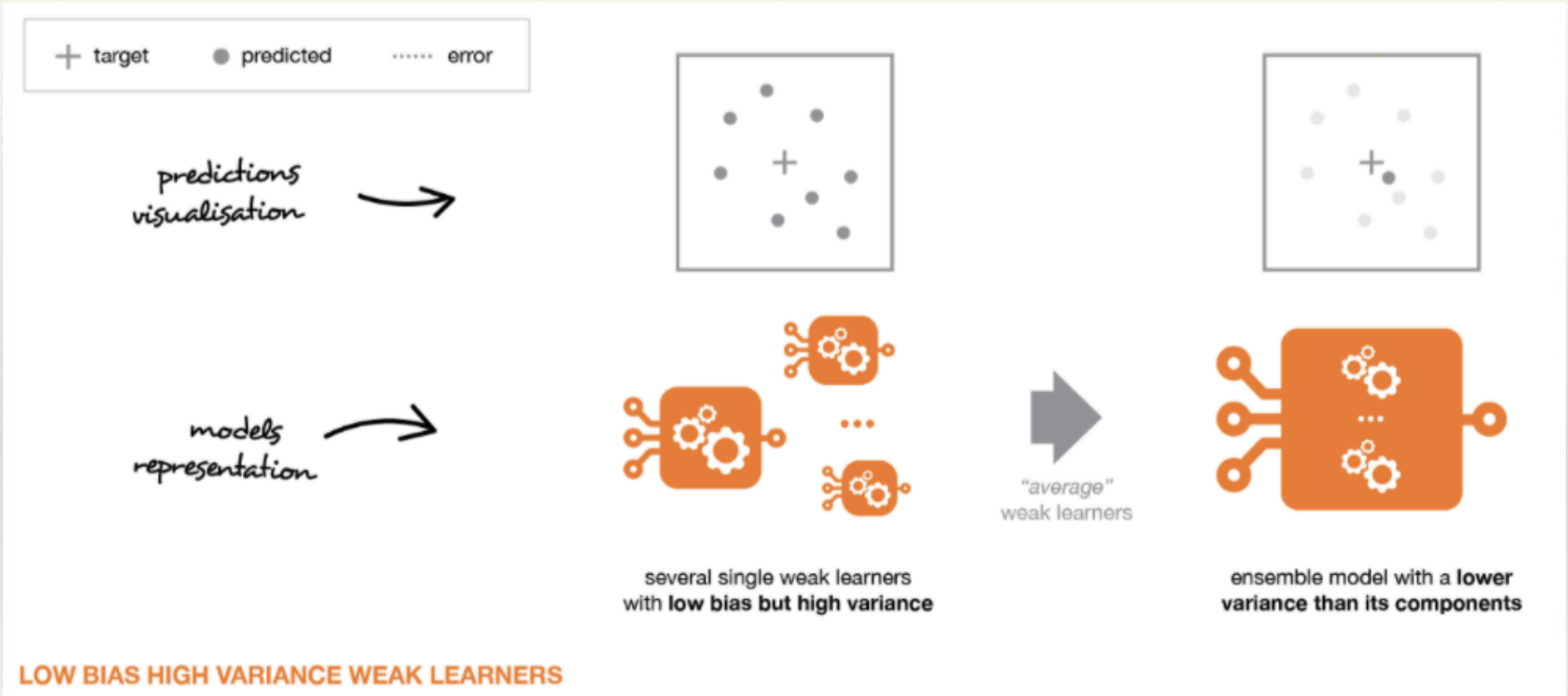
- n_estimators (and early stopping) - i.e. how many subtrees will be trained
- max_depth - i.e. the maximum tree depth each individual tree can grow to
- learning rate
- reg_alpha and reg_lambda - i.e. control the L1 and L2 regularisation terms
- …

Limit them, otherwise any grid / random search with all of the hyperparameters XGBoost allows you to tune would quickly make the search space explode. Start with few, and then dive into the others only if you still have trouble with overfitting.

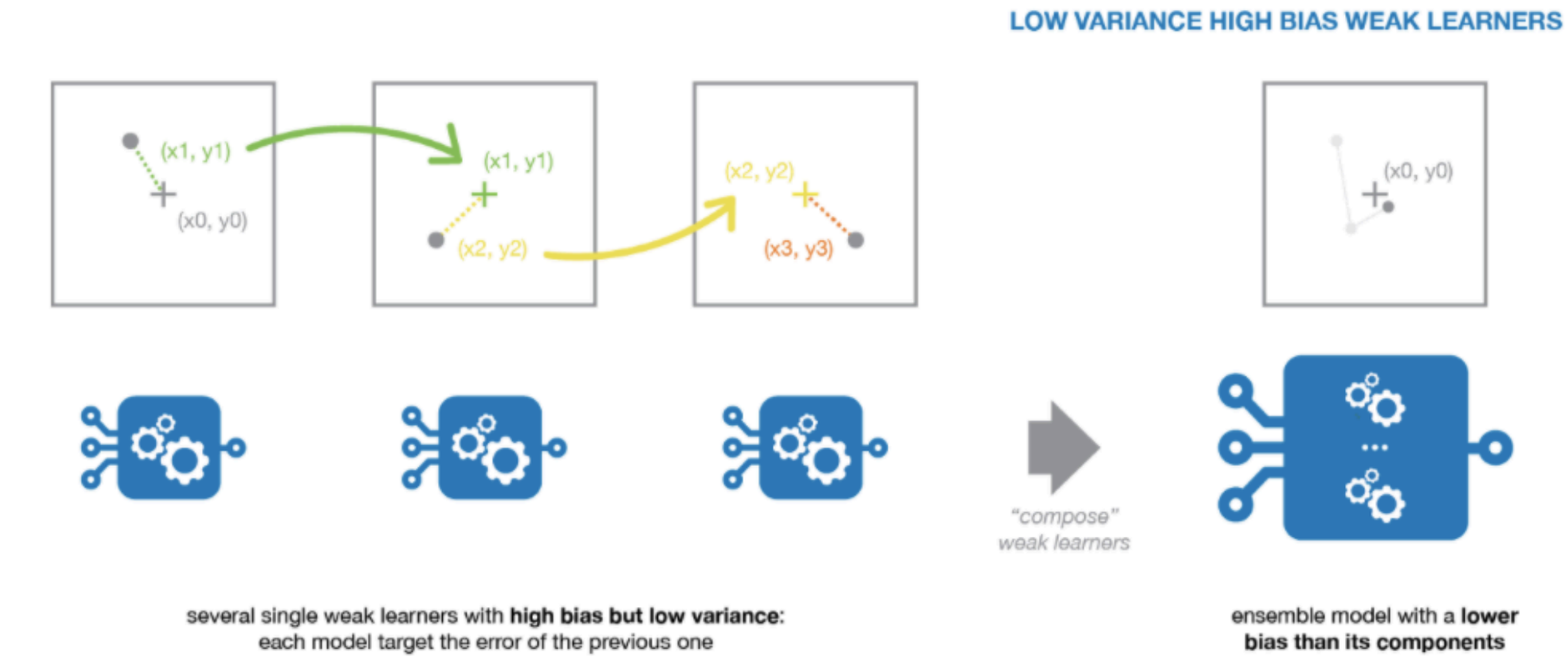
A visual summary (bagging vs boosting)

How many base, weak learners?

How to deal with weak learners? **Low-bias, high-variance**

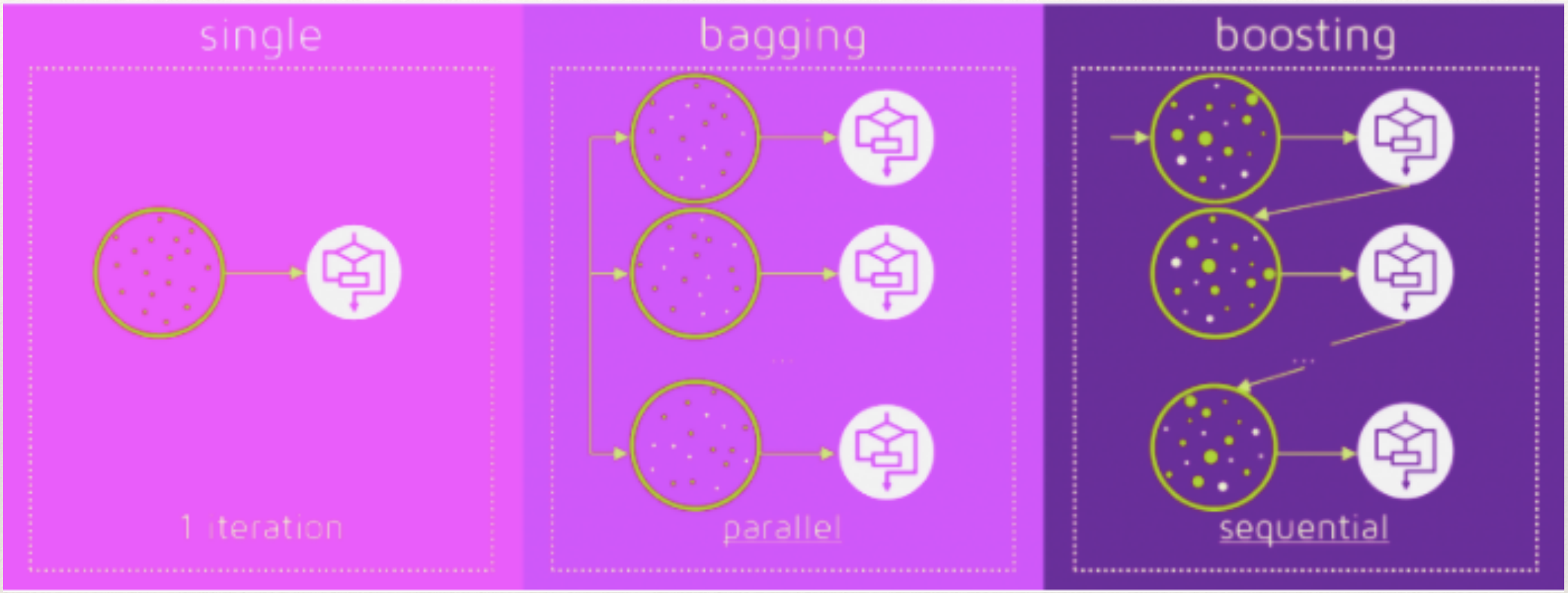


How to deal with weak learners? **Low-variance, high-bias**

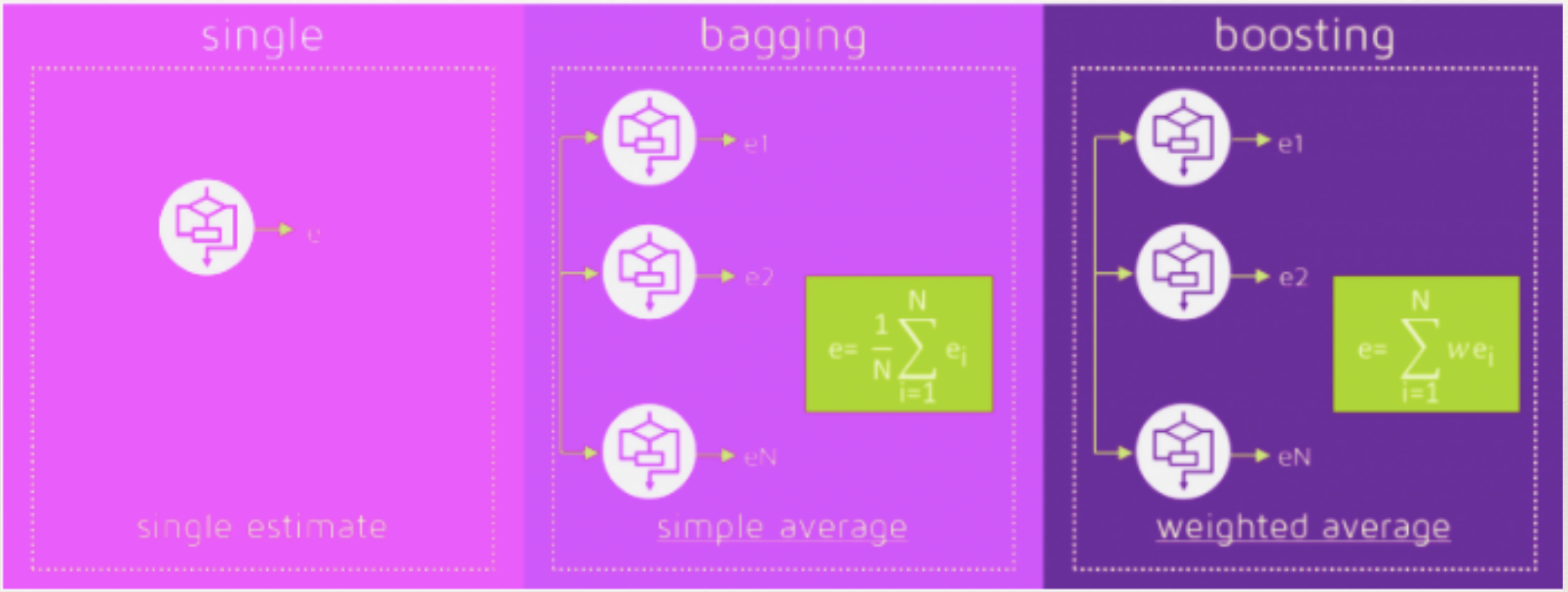


How do you use the training dataset?

How do you do the training?



How do you perform the classification?



Ensemble :: Voting

Ensemble :: Bagging

Ensemble :: Boosting

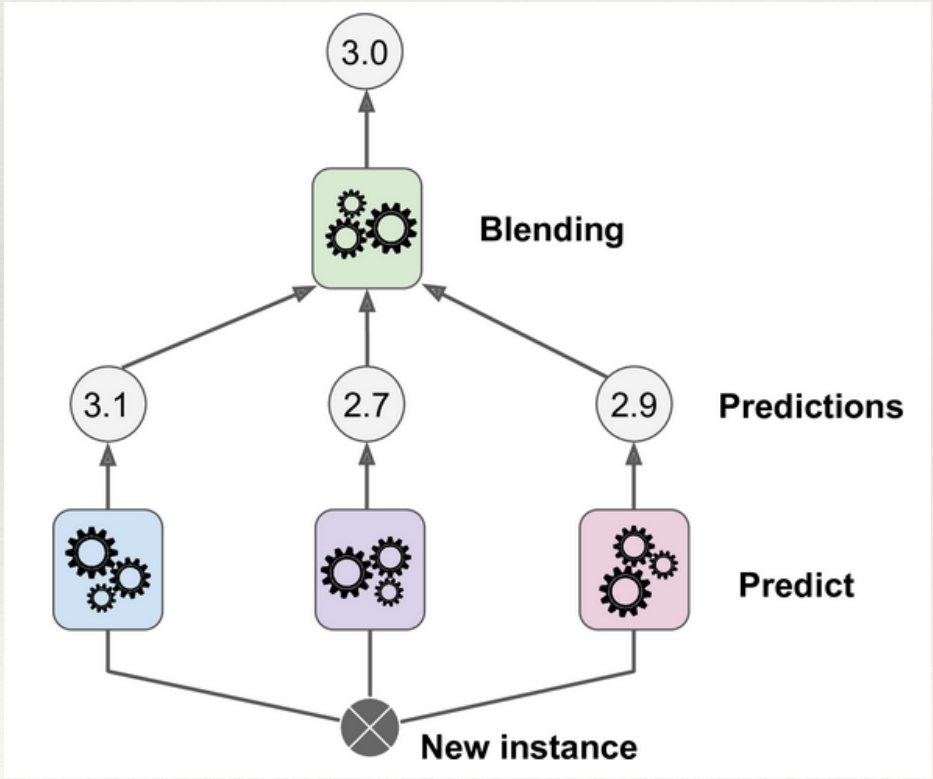
Ensemble :: Stacking

Stacking

Stacking (short for "stacked generalisation") is another Ensemble method

Idea: aggregate predictions from predictors in the ensemble not using trivial functions (such as hard voting) but **by training a model to perform such aggregation**.

- The additional intelligence lies in a final predictor (**blender**, or **meta-learner**) whose task is to take intermediate predictions as inputs and to make the final prediction



Example: **one layer**
(but you can have more)

Stacking (***in words***)

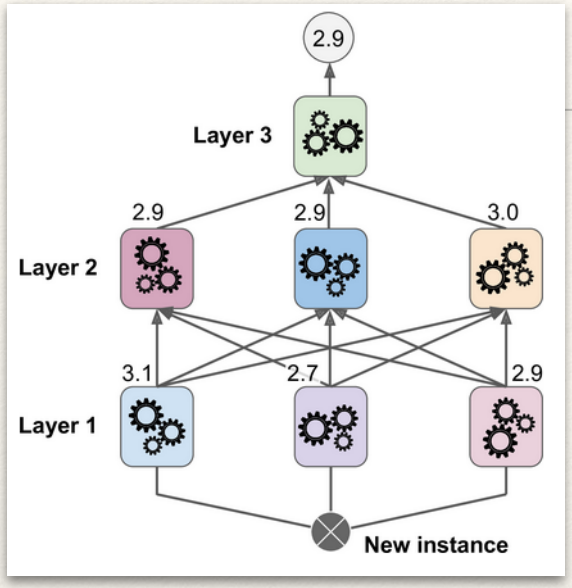
The trick is to split the training set into subsets:

- the first one is used to train the first layer
- the second one is used to create the training set used to train the second layer (using predictions made by the predictors of the first layer)
- the third one is used to create the training set to train the third layer (using predictions made by the predictors of the second layer)

Once this is done, we can make a prediction for a new instance by going through each layer sequentially.

Clear? No? Let's go visual!

Stacking (*in pictures*) → start from the bottom..

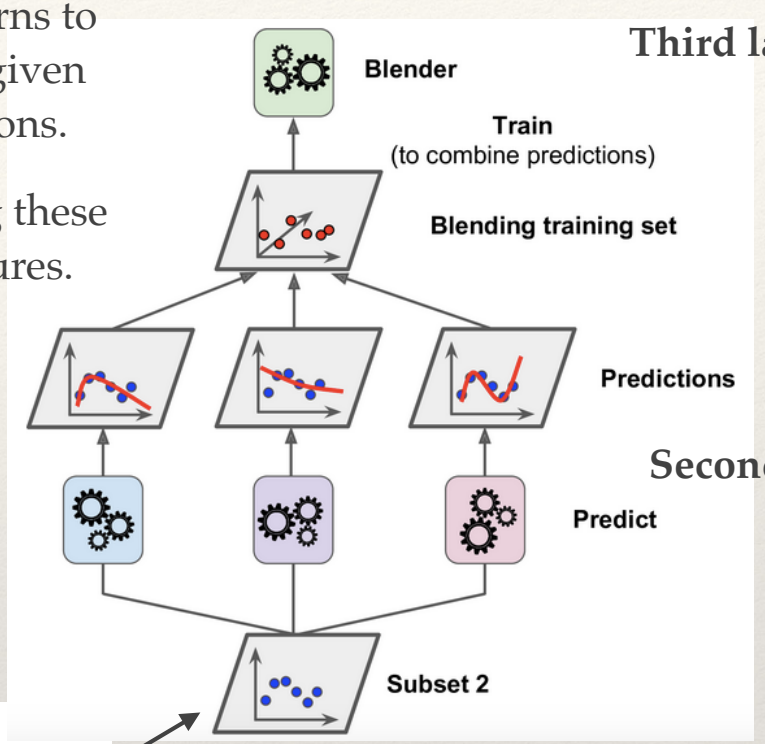


The **blender** is trained on this new training set, so it learns to predict the target value given the first layer's predictions.

I create **a new training set** using these predicted values as input features.

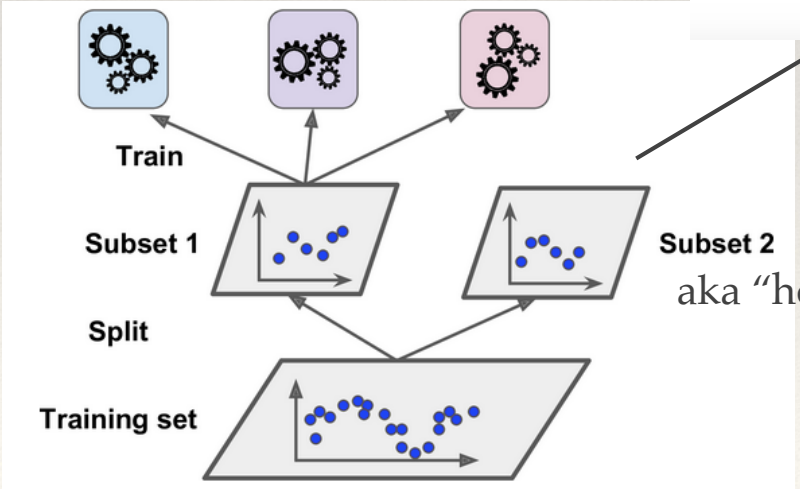
For each instance in Subset 2, I get 3 predicted values

First layer predictors are used to make predictions on Subset 2 (held-out, never seen, so predictions are "clean")



First layer

Subset 1 is used to **train** the predictors in the first layer



aka "hold-out set"

Applied Machine Learning - Advanced

Prof. Daniele Bonacorsi

Hands-on: Ensemble learning

Data Science and Computation PhD + Master in Bioinformatics
University of Bologna

Hands-on:

Ensemble learning

[credits to: A. Geron, "Hands-On Machine Learning With Scikit-Learn and Tensorflow"]

Coding material

..HandsOn_Ensemble.ipynb

→ gym on the notebook

Processing Sequences

(with RNNs and more..)

Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (**RNN**s) are a class of NNs that can "predict future behaviours" (*well.. up to a point, of course!*).

Examples:

- analyse time series data e.g. stock prices, and (hope to) tell when to buy or sell
- in autonomous driving systems, (we hope) they can anticipate car trajectories and avoid accidents
- … (in your scientific domain, **for sure** you have in mind examples of time series..)

More concretely: all NNs considered so far worked with fixed-sized inputs. RRNs instead can work on sequences of arbitrary lengths

- e.g. they can take as input numerical data, sentences, audio samples..

RNNs are extremely useful also for **natural language processing** (**NLP**) systems, such as automatic translation or speech-to-text

About RNNs

We will look at:

- the fundamental concepts underlying RNNs
- how to train them using back-propagation through time
- we will use them to forecast a time series
- we will explore the main difficulties that RNNs face:
  - a digression on **training large and deep NNs in general,** and related issues, impacting also RNNs
  - discussion of issues specific to RNNs, e.g. dealing with long sequences and the issues of a (very) limited short-term memory

About RNNs

First of all: RNNs are *not* the *only* type of NNs capable of handling sequential data:

- for small sequences, a **regular dense network** can do the trick…
- … and for very long sequences, such as audio samples or text, **CNNs** can actually work quite well too.

We will discuss both these possibilities, alongside with RNNs (and similar architectures)

A disclaimer before we start

The example and plots we use for this lecture is more "synthetic" than taken from real-world exercises..

- *"educational choice": intended to be more explanatory in my view, but also limited as it is not a real application*

Recurrent Neurons and Layers

The core idea behind RNNs

Up to now we have focused on feedforward NNs

- where the activations flow only in one direction, from the input layer to the output layer

A RNN looks very much like a feedforward NN, except **it also has connections pointing backward**.

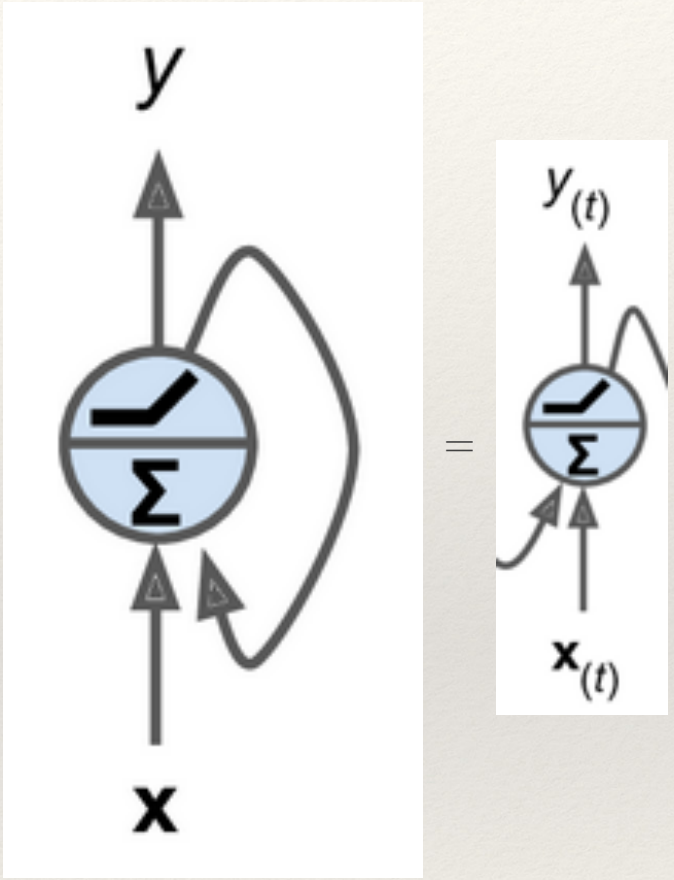
Let's see how this works.

Recurrent neuron

The simplest possible RNN is composed of one neuron receiving inputs, producing an output, and sending that output back to itself.

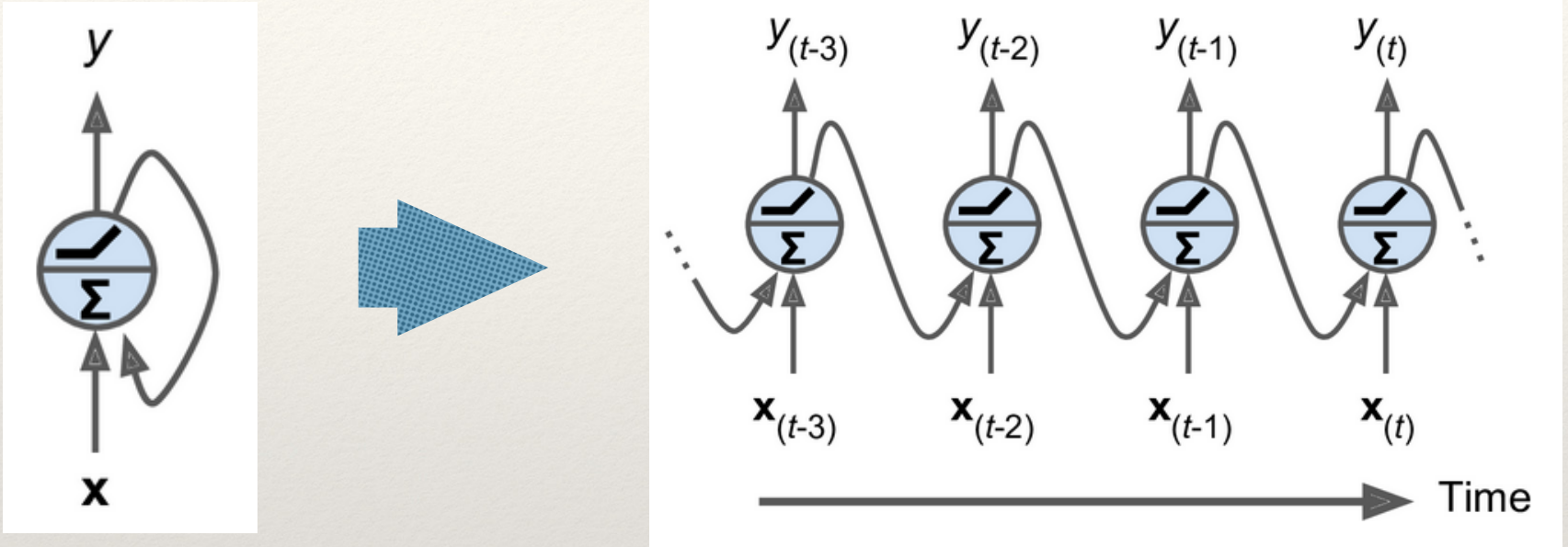
At each time step **t** (also called a **frame**), this recurrent neuron receives:

- the inputs **x(t)**
- its own output from the previous time step, **y(t–1)**
  - Since there is no previous output at the first time step, initially this is generally set to 0



"Unrolling through time"

We can represent this tiny network against the time axis (see below)



A single recurrent neuron..

.. unrolled through time (right)

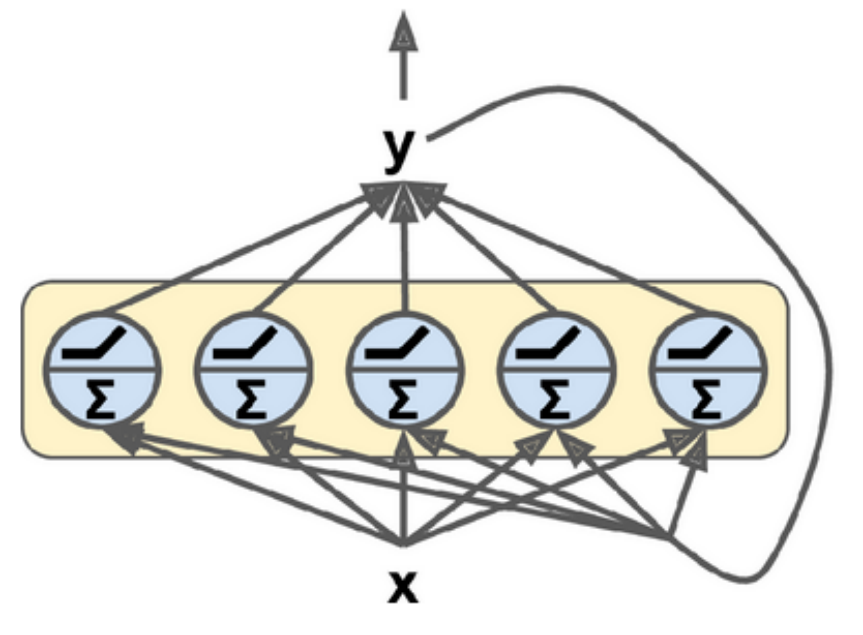
This is not the actual arch display, this is just how we see it to understand it properly, via "**unrolling the network through time**"

- it is the *same* recurrent neuron, just represented once per time step
- note that *input and output are scalars* here.. see next

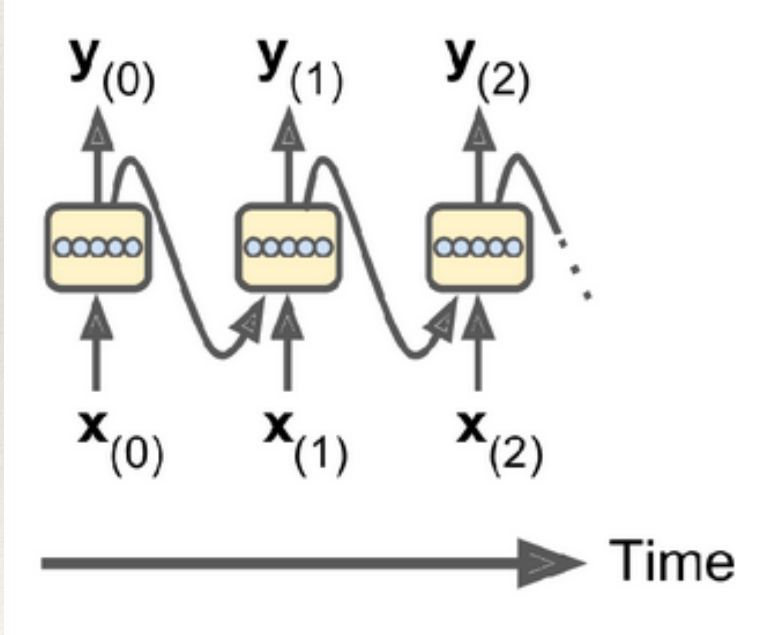
Layer of recurrent neurons

You can easily create a **layer** of recurrent neurons.

- the layer, at each time step t, receives both the input vector (not scalar) **x(t)** and the output vector (not scalar) from the previous time step **y(t–1)**
  - Note that *both the inputs and outputs are vectors* now (when there was just a single neuron, the output was a scalar; now we have a layer of such recurrent neurons, so scalars become vectors)



A layer of recurrent neurons..



.. unrolled through time

Weights

Each recurrent neuron has two sets of weights (weight vectors) - as there are 2 types of input:

- one for the inputs **x(t)** - call it $\mathbf{w_x}$ (same as before)
- one for the input which is the output from previous time step, **y(t–1)** - call it $\mathbf{w_y}$

If we consider the whole recurrent layer instead of just one recurrent neuron, we can place all the weight vectors in two weight matrices, call them $\mathbf{W_x}$ and $\mathbf{W_y}$

The output vector of the whole recurrent layer can then be computed pretty much as you might expect (see below)

- **b** is the bias vector and **ϕ(·)** is the activation function (e.g. ReLU)

$$\mathbf{y}_{(t)} = \phi\left(\mathbf{W}_x{}^T\mathbf{x}_{(t)} + \mathbf{W}_y{}^T\mathbf{y}_{(t-1)} + \mathbf{b}\right)$$

Output of a recurrent layer for a single instance

NOTE: many researchers prefer to use tanh in RNNs rather than ReLU (see e.g. [Ref-Pham], [Ref-Le])

Just as with feedforward neural networks, we can compute a recurrent layer's output in one shot for a whole mini-batch by placing all the inputs at time step t in an input matrix **X**(t):

$$\mathbf{y}_{(t)} = \phi\left(\mathbf{W}_x{}^T\mathbf{x}_{(t)} + \mathbf{W}_y{}^T\mathbf{y}_{(t-1)} + \mathbf{b}\right)$$

Output of a recurrent layer for a **single** instance

$$\mathbf{Y}_{(t)} = \phi\left(\mathbf{X}_{(t)}\mathbf{W}_x + \mathbf{Y}_{(t-1)}\mathbf{W}_y + \mathbf{b}\right)$$

$$= \phi\left(\left[\mathbf{X}_{(t)} \quad \mathbf{Y}_{(t-1)}\right]\mathbf{W} + \mathbf{b}\right) \text{ with } \mathbf{W} = \begin{bmatrix}\mathbf{W}_x \\ \mathbf{W}_y\end{bmatrix}$$

Outputs of a layer of recurrent neurons for **all instances** in a mini-batch

$$\mathbf{Y}_{(t)} = \phi\left(\mathbf{X}_{(t)}\mathbf{W}_x + \mathbf{Y}_{(t-1)}\mathbf{W}_y + \mathbf{b}\right)$$

$$= \phi\left(\left[\mathbf{X}_{(t)} \quad \mathbf{Y}_{(t-1)}\right]\mathbf{W} + \mathbf{b}\right) \text{ with } \mathbf{W} = \begin{bmatrix}\mathbf{W}_x \\ \mathbf{W}_y\end{bmatrix}$$

In this equation:

- **m** is the number of instances in the mini-batch
- $\mathbf{n_{neurons}}$ is the number of neurons
- $\mathbf{n_{inputs}}$ is the number of input features)
- **Y(t)** is a $m \times n_{neurons}$ matrix containing the layer's outputs at time step t for each instance in the mini-batch
- **X(t)** is an $m \times n_{inputs}$ matrix containing the inputs for all instances
- $\mathbf{W_x}$ is an $n_{inputs} \times n_{neurons}$ matrix containing the connection weights for the inputs of the current time step
- $\mathbf{W_y}$ is an $n_{neurons} \times n_{neurons}$ matrix containing the connection weights for the outputs of the previous time step
- **b** is a vector of size $n_{neurons}$ containing each neuron's bias term
- The weight matrices $\mathbf{W_x}$ and $\mathbf{W_y}$ are often concatenated vertically into a single weight matrix **W** of shape $(n_{inputs}+n_{neurons}) \times n_{neurons}$
- The notation **[X(t) Y(t–1)]** represents the horizontal concatenation of the matrices **X(t)** and **Y(t–1)**

Note that Y(t) is a function of X(t) and Y(t–1), which is a function of X(t–1) and Y(t–2), which is a function of X(t–2) and Y(t–3), and so on…

This makes Y(t) a function of all the inputs since time t = 0 (that is, X(0), X(1), …, X(t)).

At the first time step, t = 0, there are no previous outputs, so they are typically assumed to be all zeros.

Memory cells

[it is intuitive that the concept of "**memory**" would have appeared at some point in this architecture..]

"Memory" cell

Since the output of a recurrent neuron at time step t is a function of all the inputs from previous time steps, you could say it has a form of "*memory*"

- (.. the biological inspiration and a taste of neurosciences still persists..)

A part of a NN that is **stateful over time**, i.e. it preserves some state across time steps, is called a **memory cell** (or simply a cell, in the following).

In this sense, *a single recurrent neuron, or a layer of recurrent neurons, acts as a very basic memory cell*, capable of learning only short patterns

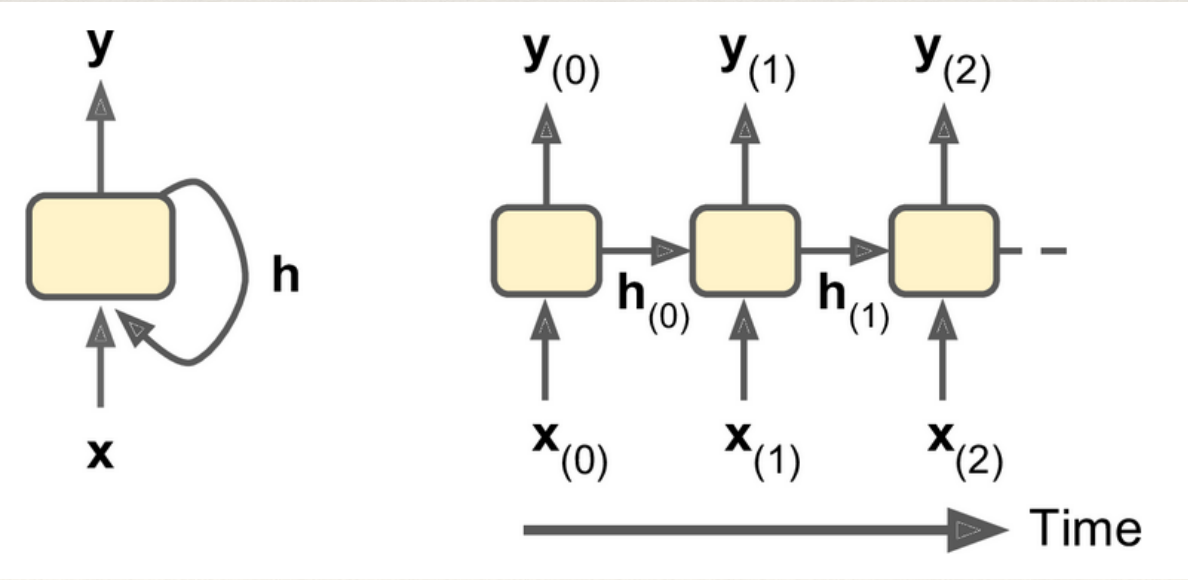
- "short" = about 10 steps long - but this varies, depending on the task
- there are more complex and powerful types of cells capable of learning longer patterns (roughly 10 times longer - but again, this depends on the task).

Formalising the "memory"

In general a cell's **state** at time step t, denoted **h(t)** (the "h" stands for "hidden"), is a function of some inputs at that time step and its state at the previous time step: **h(t) = f(h(t–1), x(t))**.

And its output at time step t, denoted **y(t)**, is also a function of the previous state, and not only the current inputs

- In the case of the basic cells we have discussed so far, the output is simply equal to the state, but in more complex cells this is not always the case..

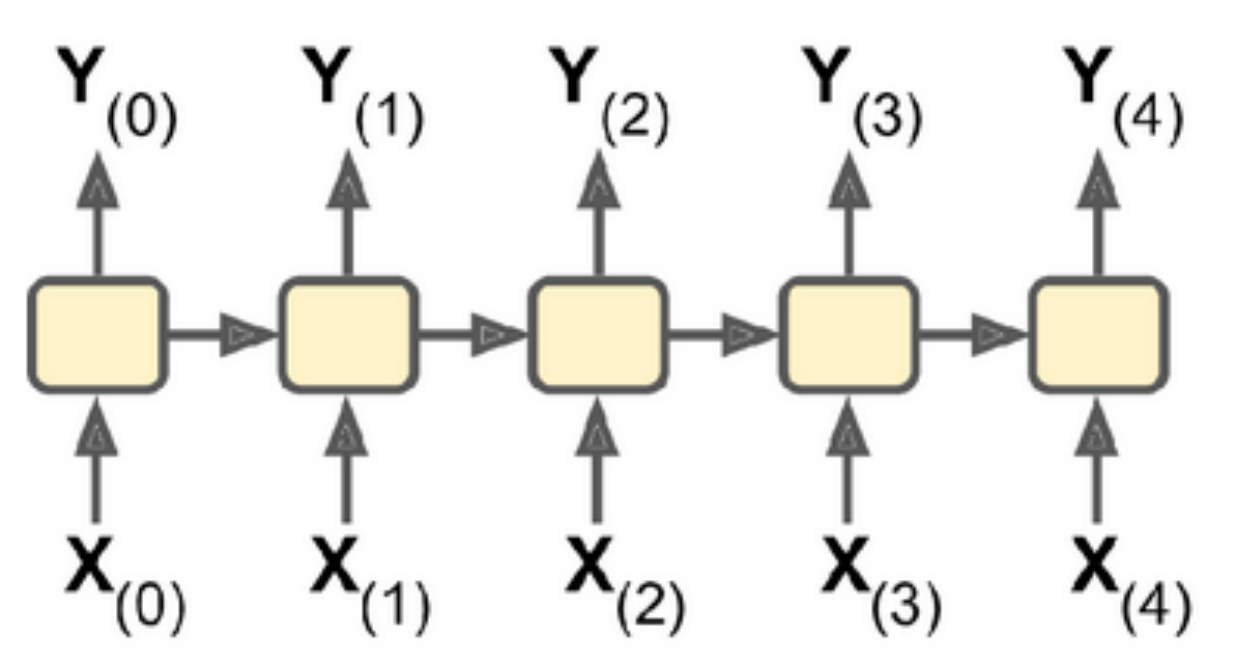


A cell's hidden state and its output may be different than these..

Input and Output Sequences

seq-2-seq

As a (nice) consequence of all this, a RNN can simultaneously *take a sequence of inputs and produce a sequence of outputs* ("**sequence-to-sequence" network**):



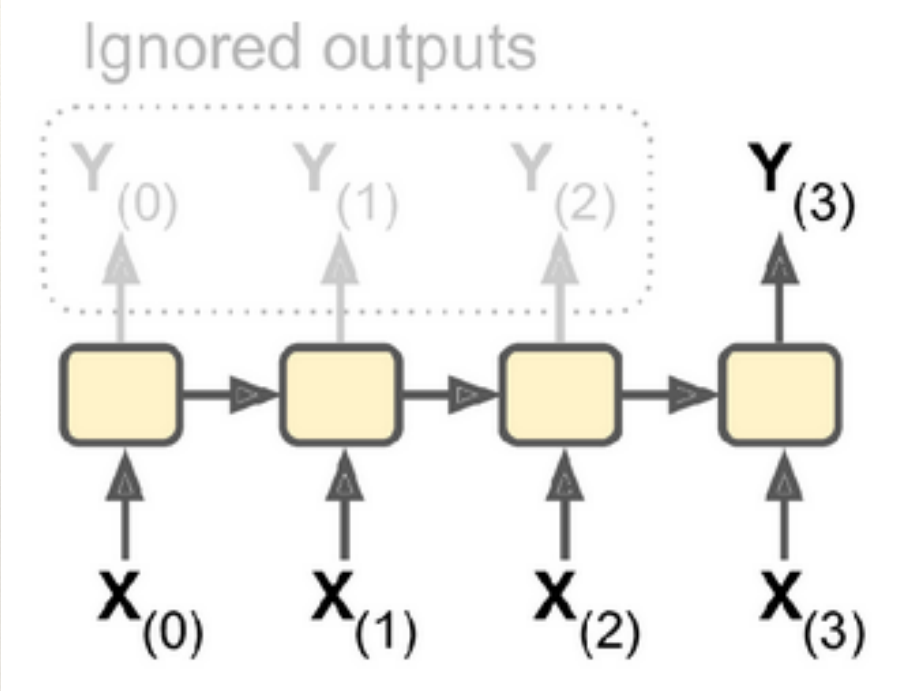
Seq to seq

This type of network is useful for **predicting time series**

E.g. stock prices: you feed it the prices over the last N days, and it must output the prices shifted by one day into the future (i.e. from N–1 days ago to tomorrow, i.e. it "predicts" tomorrow)

seq-2-vec

Alternatively, you could feed the network a sequence of inputs and ignore all outputs except for the last one ("**sequence-to-vector**" **network**):

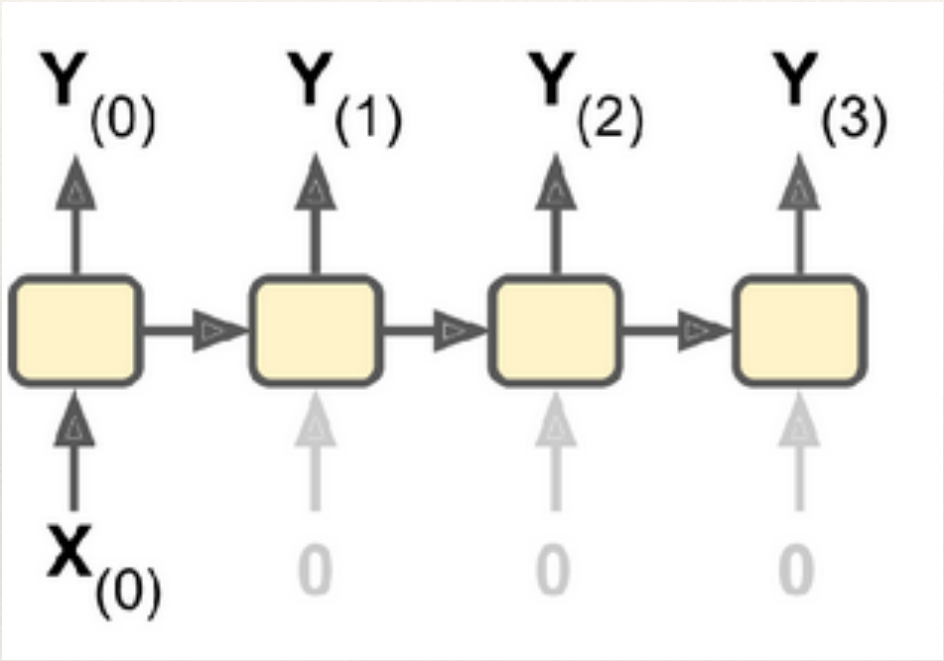


Seq to vec

E.g. you could feed the network a sequence of words corresponding to a movie review, and the network could output a sentiment score (e.g. from –1 [hate] to +1 [love]).

vec-2-seq

Conversely, you could feed the network the same input vector over and over again at each time step and let it output a sequence ("**vector-to-sequence" network**):

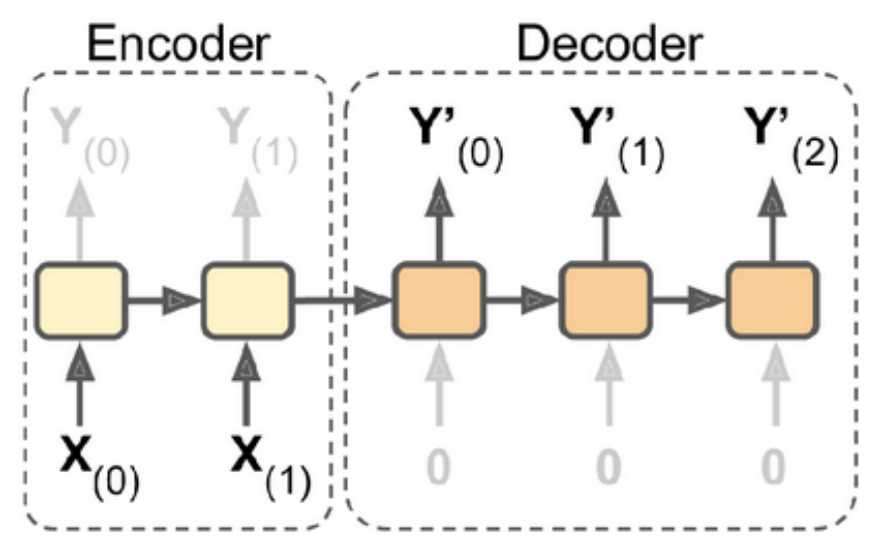


Vec to seq

E.g. the input could be an image (or the output of a CNN), and the output could be a caption for that image

Encoder-Decoder

Lastly, you could have a *sequence-to-vector network*, called an **encoder**, followed by a *vector-to-sequence network*, called a **decoder**:



Encoder/Decoder

E.g. this can be used for translation of a sentence from one language to another

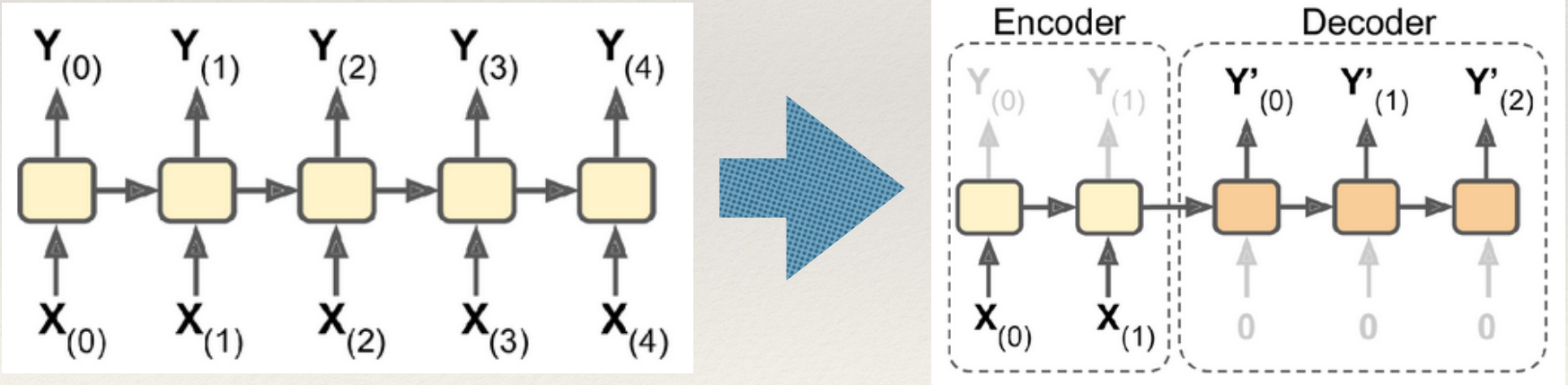
- You would feed the network a sentence in one language, the encoder would convert this sentence into a single vector representation, and then the decoder would decode this vector into a sentence in another language

Encoder-Decoder

Why don't I use a seq-2-seq instead?

This two-step model, called an **Encoder-Decoder**, works much better than trying to translate on the fly with a single *sequence-to-sequence RNN*

- the last words of a sentence can affect the first words of the translation, so you need to wait until you have seen the whole sentence before translating it.



All this sounds promising - and a for a variety of concrete use-cases (some were already mentioned in the slides as examples).

How do you train a RNN?